

Protocol: архитектура по этапам

Навигатор клинических протоколов Минздрава РБ и ориентировочная проверка консультативных заключений (КЗ).

Документ описывает текущую реализацию (MVP), целевую интеграцию с МИС и Минздравом через стандарт FHIR BY / ЦИСЗ, а также предложения по масштабированию на тысячи КЗ в сутки.

Версия документа: 2026-06-02 · Сборка приложения: 2026-06-01-r59 · Не является ТЗ и не заменяет нормативные акты Минздрава.

1. Назначение и границы системы

MVP Protocol - веб-сервис и API для двух сценариев:

1. **Поиск протоколов** - подбор официальных КП Минздрава РБ по диагнозу, МКБ-10, жалобам и рубрикам каталога.
2. **Проверка КЗ** - ориентировочная сверка консультативного заключения с фрагментами протоколов и формализованными правилами; отчёт для врача и методслужбы.

Не входит: установление диагноза, замена очного приёма, МЭЭ, юридически обязательная экспертиза, прямая запись в медицинскую карту (на этапе пилота).

Два контура intelligence: (1) **LLM** - языковая модель для синтеза, уточнения запроса и дополнения ответа; (2) **детерминированный слой** `clinical_knowledge/` - парсинг, правила из протоколов, scoring без «магии» модели.

2. Этап 0. Корпус протоколов и справочники

2.1. Источник данных

Официальный каталог PDF на `minzdrav.gov.by` (раздел «Стандарты обследования и лечения»). Синхронизация скриптом `download_minzdrav_protocols.py` в каталог `minzdrav_protocols/` (~478 PDF, 24 рубрики).

2.2. Индекс каталога

`index.csv` - метаданные: рубрика, год в имени файла, признак постановления МЗ. Используется для UI-бейджей, статистики презентации и фильтрации.

2.3. Справочники

- `protocols.json`, `protocol_meta.json` - реестр путей и метаданные.
- `data/catalog/` - 85 нозологий, 482 автоизвлечённых правила, отчёт покрытия 100% PDF.
- `data/protocol_summaries/` - YAML/JSON карточки протоколов (~465 summaries) для навигации и расширения правил.
- `config/` - требования к КЗ, веса scoring, red flags.

3. Этап 1. Подготовка корпуса для RAG

3.1. Два пути сборки (legacy и новый)

Путь	Скрипты	Результат
Legacy	<code>extract_corpus.py</code> → <code>build_chunks.py</code> → <code>split_chunks_jsonl.py</code>	<code>corpus_chunks_parts/*.jsonl</code>
Новый	<code>corpus_pipeline/run_pipeline</code>	Секции, таблицы, entities → <code>output/chunks/</code>

3.2. Чанки RAG

~105 000 фрагментов текста с разметкой разделов (диагноз, лечение, обследование). Хранятся в `corpus_chunks_parts/` (JSONL, 7 частей). При старте сервера загружаются в память фоновым потоком.

3.3. Индексы поиска

При загрузке строятся: лексический инвертированный индекс, BM25 (`retrieval_bm25.py`), маршрутизация по симптомам (`symptom_routing.json`).

4. Этап 2. Слой clinical_knowledge (детерминированный)

Каталог `clinical_knowledge/` (~61 модуль). Публичный API: `clinical_knowledge_status`, `match_protocol_cards`, `run_rule_checker`, `extract_consult_facts_heuristic`.

Блок	Модули	Функция
Схемы	<code>consult_schema.py</code>	ConsultationDocument, ComplianceReport, 8 блоков scoring
Парсинг КЗ	<code>consult_parser.py</code> , <code>text_extract.py</code> , <code>medication_parser.py</code>	PDF/TXT/DOCX → структура документа
Подбор КП	<code>protocol_match.py</code> , <code>diagnosis_icd.py</code>	Сопоставление с протоколами по МКБ и рубрике
Правила	<code>rule_checker.py</code> , <code>rules_from_path.py</code>	482 правила из PDF, покрытие по рубрикам
Compliance	<code>compliance_engine.py</code> , <code>requirement_checker.py</code> , <code>safety_checker.py</code>	Итоговый отчёт, issues, source_refs
Summaries	<code>protocol_summary/</code>	Карточки протоколов → rules → RAG chunks
Отчёты	<code>consult_report.py</code> , <code>batch_runner.py</code>	HTML/MD для печати, пакетная обработка

Оркестратор: `consult_analysis.analyze_consultation_text()`. В production вызывается из `consult_review_pipeline.py`.

5. Этап 3. Runtime: rag_server.py (FastAPI)

5.1. Стек

Python 3.11+, FastAPI, uvicorn. Учётные данные для вызова языковой модели задаются только в конфигурации сервера (переменные окружения или защищённое хранилище секретов) и не передаются в браузер.

5.2. Загрузка при старте

Фоновый поток `load_data()` : JSONL-чанки, protocols, meta, summary chunks, BM25. Endpoint `/health` отвечает до полной готовности RAG (важно для health-check при облачном развёртывании и в Kubernetes).

5.3. Гибридный retrieve

Функция `retrieve()` : лексика + BM25 + routing boost + фильтр аудитории + опциональный embed-rerank через LLM. Результат - топ фрагментов по протоколам.

5.4. Группы API

Домен	Endpoints
Health	GET <code>/health</code> , <code>/api/version</code> , <code>/api/corpus-stats</code>
Поиск	POST <code>/api/assist</code> , <code>/api/protocol-detail</code> , <code>/api/protocol-practical</code> , <code>/api/kz-matrix</code> , GET <code>/api/protocol-pdf</code>
КЗ	POST <code>/api/consult-review</code> , <code>/api/consult-review/stream</code> (SSE)
Справочники	<code>/api/specialties</code> , <code>/api/icd-suggest</code> , <code>/api/training-cases</code>
Аналитика	<code>/api/presentation-stats</code> , <code>/api/quality-benchmark</code> , <code>/api/pilot-analytics-demo</code>
Clinical knowledge	<code>/api/clinical-knowledge/status</code> , <code>/build-status</code> , <code>/benchmark</code>

6. Этап 4. Проверка КЗ (pipeline)

Модуль `consult_review_pipeline.py` - оркестрация с SSE-прогрессом для UI.

PDF/TXT/DOCX

- извлечение текста
- МКБ-10 + якорь рубрики врача
- retrieve (RAG по allowlist протоколов)
- match_protocol_cards + run_rule_checker
- build_compliance_report (детерминированно)
- LLM: синтез критериев и дополнение (если включено)
- structured_analysis + report_html + report_markdown

Флаг `CONSULT_STRUCTURED_ANALYSIS=1` включает полный структурный разбор. Fast mode на production сокращает второй проход RAG и таймауты.

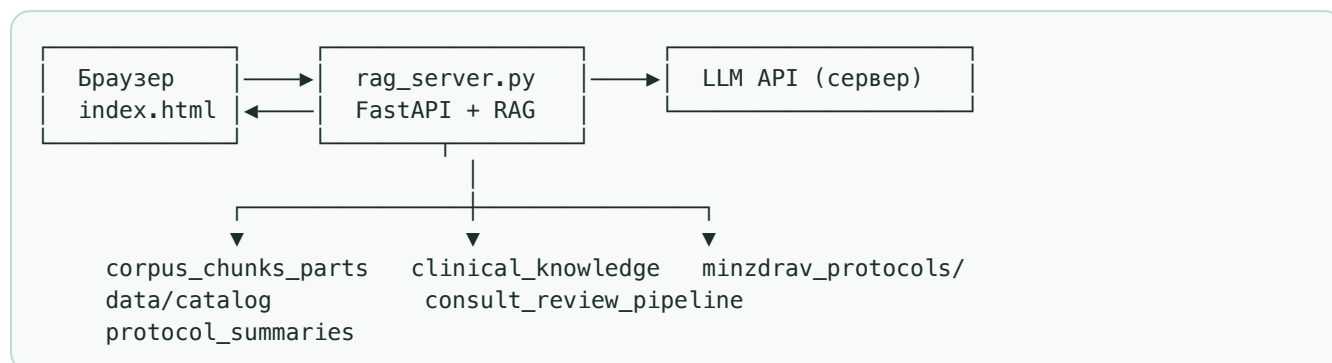
7. Этап 5. Frontend (index.html)

Монолитный SPA (~12k строк): inline CSS + vanilla JS, без bundler.

- **Вкладка по умолчанию:** «Проверка КЗ» (`#consult-review`).
- **Вторая вкладка:** «Поиск протоколов» (`#search`).
- Демо: `?presentation=1` - overlay для показа руководству.
- SSE для потоковой проверки КЗ; lazy-load развёрнутых выдержек протоколов.
- Читаемые ссылки на PDF протоколов (`beautify_protocol_title`).

Подробная архитектура проверки КЗ, оценок, подбора протоколов и интеграции FHIR/ЦИСЗ: [architecture-kravira-fhir-mis-print.html](#) (печать в PDF). Также: `docs/mvp-presentation.html` , `docs/pre-sign-checklist-print.html` .

8. Сводная схема текущей архитектуры



8.1. Поток «Поиск протоколов»

Запрос → МКБ (`icd_mkb.py` + LLM) → retrieve → LLM формирует список протоколов с confidence → опционально protocol-detail.

8.2. Поток «Проверка КЗ»

Файл → pipeline → детерминированный compliance + RAG + LLM → отчёт с цитатами из официальных PDF.

9. Будущая интеграция с МИС и Минздравом

Этап 2 Согласно [FHIR BY](#) и [регламенту МИС ОЗ - ЦИСЗ](#) (v1.4, FHIR 5.0.0):

9.1. Разделение ответственности

Система	Что проверяет
ЦИСЗ	Профили FHIR, структура Bundle, справочники НСИ, OAuth 2.0, импорт пакета
Protocol	Соответствие <i>содержания</i> КЗ клиническим протоколам Минздрава (ориентир, не МЭЭ)
МИС	Формирование Bundle, ЭЦП, UI врача, вызов Protocol до подписи
Минздрав	Владелец корпуса КП, методические требования, пилоты, агрегированная аналитика

9.2. Точка встраивания в МИС

Врач заполняет КЗ в МИС

- МИС собирает FHIR Bundle (Patient, Encounter, Condition, Composition...)
- [НОВОЕ] вызов Protocol API (до ЭЦП)
- предупреждения / отчёт врачу
- врач исправляет (при необходимости)
- ЭЦП (ЕСИФЮЛ) → импорт в ЦИСЗ (202 Accepted → poll status)

9.3. Маппинг FHIR → Protocol

FHIR BY	Protocol
Patient	PatientContext (пол, возраст)
Condition + МКБ-10	диагнозы, match_protocol_cards
Observation	витальные показатели в контексте КЗ
Encounter	care_setting, применимость протокола
Composition	секции КЗ → consult_parser
DocumentReference / PDF	текущий POST /api/consult-review

9.4. Планируемые фазы

Фаза	Срок (ориентир)	Содержание
A	3-6 мес.	REST из МИС: МКБ + текст → <code>/api/consult-review</code> ; кнопка «Проверить по протоколу»
B	6-12 мес.	Адаптер <code>fhir_bundle_to_consult</code> ; endpoint <code>POST /api/fhir/consult-compliance</code> ; ответ в духе OperationOutcome

Фаза	Срок (ориентир)	Содержание
С	12+ мес.	On-premise в контуре РБ; дашборд Минздрава (агрегаты без ПДн); синхронизация корпуса КП

9.5. Развёртывание

См. `docs/deployment-belarus.md` : сервер в контуре оператора/Минздрава, корпус на диске/S3, LLM - разрешённый API или on-premise модель в РБ, ПДн не выносить в публичный интернет.

10. Масштабирование: тысячи КЗ в сутки

При полной проверке каждого КЗ с LLM + RAG нагрузка растёт линейно. Ниже - меры ускорения и снижения стоимости.

10.1. Технические оптимизации

Мера	Эффект
Детерминированный fast path	Сначала rule_checker + requirement_checker без LLM; LLM только при пограничных баллах или по запросу
Кэш consult-review	<code>CONSULT_REVIEW_CACHE</code> - повторные проверки того же текста/хеша
Очередь + воркеры	Celery/RQ: асинхронная обработка пакетов; API возвращает job_id, МИС опрашивает статус (аналог poll ЦИС3)
Отключение embed-rerank	На batch-режиме: только lex + BM25 (уже есть флаг в CI)
Предварительный digest	Сокращение текста КЗ до ключевых секций перед LLM (уже частично в pipeline)
Горизонтальное масштабирование	Несколько инстансов API за балансировщиком; корпус read-only на shared disk
Инкрементальный corpus	Не перезагружать весь RAG при деплое - hot-reload index.csv

10.2. Ориентиры производительности (пилот)

- Детерминированный разбор одного КЗ (без LLM): секунды.
- Полная проверка с LLM: десятки секунд - минуты (зависит от длины и fast mode).
- Целевой batch: 100% КЗ - только детерминированный скоринг; 5-15% - углублённо с LLM.

11. Семантический RAG и пять фаз ускорения (реализовано)

Аддитивный контур поверх существующего lex+BM25+rules. Детерминированный scoring не заменяется.

Фаза	Модули / скрипты	Эффект
1	<code>scripts/build_chunk_embeddings.py</code> , <code>scripts/corpus_manifest.py</code> , <code>RAG_PRECOMPUTED_CHUNK_EMBED</code>	Офлайн-эмбединги чанков; один API-вызов на запрос вместо пула ~44
2	<code>clinical_knowledge/vector_index.py</code> , <code>scripts/build_vector_index.py</code> , <code>RAG_VECTOR_INDEX</code>	Vector top-200 → lex/BM25 → embed-rerank top-44; FAISS или numpy
3	<code>semantic_rule_fallback.py</code> , <code>RULE_SEMANTIC_FALLBACK</code>	Синонимы/токены для <code>required_exam</code> и <code>keyword_presence</code> после substring
4	<code>consult_tiering.py</code> , <code>POST /api/consult-review/tier</code>	L0/L1/L2 для массового потока МИС без полного LLM на 100% K3
5	<code>scripts/enrich_protocol_rules_batch.py</code> , <code>data/catalog/enrichment/</code>	Batch LLM → structured rules (<code>required_exam</code> , red flags) в <code>rule_checker</code>

11.1. Порядок развёртывания корпуса

1. Собрать смысловые чанки: `corpus_pipeline/run_pipeline.py` → `split_chunks_jsonl.py`
2. Офлайн embeddings: `python3 scripts/build_chunk_embeddings.py`
3. Индекс: `python3 scripts/build_vector_index.py` (опционально `pip install -r requirements-vector.txt`)
4. Манифест: `python3 scripts/corpus_manifest.py`
5. Prod env: `RAG_PRECOMPUTED_CHUNK_EMBED=1` , `RAG_VECTOR_INDEX=1`

11.2. Eval

`eval/search_quality_eval.py --embed-on --precomputed-on --vector-on` - сравнение с golden queries.

12. Методика выборочного контроля качества КЗ

Цель: выявить врачей/подразделения с систематически слабыми КЗ без проверки каждого документа LLM.

12.1. Уровни выборки

Уровень	Охват	Метод
L0 - скрининг	100% КЗ	<code>POST /api/consult-review/tier tier=L0</code> или <code>/api/consult-compliance-screen</code> . < 1 с, без LLM/RAG.
L1 - выборка	10-20% КЗ	<code>tier=L1</code> : полный <code>analyze_consultation_text</code> + compliance, без LLM-критериев.
L2 - углублённо	2-5% КЗ	<code>tier=L2</code> или <code>/api/consult-review</code> : RAG + LLM + structured hybrid score.
L3 - ручной аудит	0.5-1%	Эксперт методслужбы по списку «красных» врачей из L0-L2.

12.2. Критерии отбора «проблемных» врачей

- Средний балл L0 за месяц < 70% при n ≥ 10 КЗ.
- Доля КЗ с критическими issues (red flags) > 15%.
- Тренд: снижение балла 3 месяца подряд.
- Выбросы: единичные КЗ < 50% при обычно нормальном профиле - в L2, не автоматическая санкция.

12.3. Агрегаты для Минздрава (без ПДн)

Учреждение, рубрика, период, средний % по блокам, доля «зелёных/жёлтых/красных» зон. Без ФИО пациентов; идентификатор врача - только внутри контура учреждения.

12.4. Хранение результатов

Таблица `kz_quality_scores` : hash текста, `author_id` (из токена МИС), `timestamp`, `L0_score`, `flags`, optional `L2_job_id`. Retention по политике ИБ.

13. Вариант: блокировка ЭЦП до нормы соответствия

Опция для МИС Не юридическое требование Protocol, а **настраиваемая политика учреждения**, согласуемая с Минздравом и юристами.

13.1. Режимы (config в МИС)

Режим	Поведение
inform	Только предупреждения; ЭЦП всегда доступна (текущий пилот).
soft_gate	При балле < порога (напр. 70%) - диалог «Подписать всё равно?» + запись override в журнал.
hard_gate	Кнопка ЭЦП неактивна, пока L0 ≥ порога или методист не снял блок (workflow).
critical_only	Блок только при critical red flags (безопасность, противопоказания).

13.2. Техническая реализация в МИС

- МИС вызывает `POST /api/consult-review?mode=fast` или лёгкий `POST /api/consult-compliance-screen` (только L0).
- Ответ: `overall_compliance_pct`, `blocks`, `critical_issues[]`, `gate_allowed: bool`.
- МИС применяет локальную политику → разрешает/блокирует вызов ЕСИФЮЛ.
- При override - запись в audit log (врач, время, причина).

13.3. Риски и смягчение

- Ложные срабатывания** - порог только по L0; спорные случаи - soft_gate.
- Юридическая нагрузка** - формулировка «ориентир методслужбы», не МЭЭ.
- Соппротивление врачей** - пилот в одном отделении, обучение, прозрачные критерии из протоколов с цитатами.

14. Тестирование и качество

- CI:** Ruff + pytest (~55 модулей), миникопнyc `tests/fixtures/chunks.mini.jsonl`.
- Eval:** `eval/golden_queries.jsonl` → `data/quality_benchmark.json` для UI.
- Regression KZ:** `tests/test_regression_kz_compliance.py`, fixtures в `tests/fixtures/consultations/`.

15. Заключение

Protocol сегодня - монолитный MVP с двумя контурами: **LLM** для поиска и синтеза и **детерминированный clinical_knowledge** для воспроизводимой проверки КЗ. Корпус привязан к официальным PDF Минздрава РБ.

Интеграция с МИС через FHIR BY логична *до подписи ЭЦП*: ЦИСЗ проверяет упаковку данных, Protocol - клиническое соответствие протоколам. Масштаб на тысячи КЗ достижим за счёт L0-скрининга, выборочного L2 и очередей; блокировка ЭЦП - опциональная политика учреждения, не обязательная часть MVP.

Protocol · Kravira · Документ сгенерирован для внутреннего использования и презентаций Минздраву/МИС. Ссылки: fhir.by, minzdrav.gov.by, репозиторий protocol (docs/roadmap-mis.md, deployment-belarus.md).